

A Machine Learning Approach to Twitter Sentiment Analysis

Model Design

CSCE 3602

Kirolous Fouty
Computer Engineering
Department
American University in Cairo
Cairo, Egypt
kirolous_fouty@aucegypt.edu

Mohamed Sabry
Computer Engineering
Department
American University in Cairo
Cairo, Egypt
momo12320@aucegypt.edu

1 Algorithm Choice

Given the feedback from the pilot study and prior phase, the recommendation is to go with a Multi-Layer Perceptron (MLP) as the primary model, enhanced with word embeddings to capture contextual information in tweets. MLPs with word embeddings can effectively handle textual data and are versatile enough to learn non-linear patterns, which may suit the nuances of sentiment analysis on Twitter.

Justification for MLP:

- **Flexibility:** MLPs can adjust to complex patterns in sentiment-rich textual data, which typically involves understanding context, emotive content, and semantic nuance.
- **Efficiency:** When paired with embeddings, MLPs can offer efficient training while achieving robust performance on sentiment tasks.
- **Compatibility with Word Embeddings:** Integrating word embeddings with MLP provides a dense and context-aware representation of words, which improves the model's ability to capture sentiment.

Cons of MLP for Sentiment Analysis:

- **Limited Contextual Awareness:** Compared to models like LSTMs or transformers, MLPs lack a mechanism to capture sequential relationships and dependencies over long contexts. This may limit performance on tweets with intricate or ambiguous sentiment.
- **Potential Overfitting:** MLPs, especially with deeper architectures, can overfit on sentiment data, as they may learn to identify specific vocabulary or patterns rather than general

sentiment. To mitigate this, dropout and regularization techniques are needed.

- **Resource-Intensive Fine-Tuning:** Fine-tuning embeddings within the MLP framework can be computationally costly, especially for large embedding spaces. This requires careful resource management, particularly if deployed on cloud services with budget constraints.

2 Model Design and Architecture

Input Layer: Accepts pre-processed tweets (either as sequences of word embeddings or an embedding layer directly integrated into the MLP). Consider a maximum tweet length, such as 280 tokens (aligned with Twitter's character limit).

Embedding Layer:

- **Embedding Source:** Pre-trained embeddings, such as GloVe, Word2Vec, provide initial word representations to capture contextual and semantic information. These embeddings should ideally be fine-tuned on the training data to adapt to the sentiment nuances specific to Twitter content.
- **Dimension of Embeddings:** The embedding layer could use 100 to 300 dimensions to balance computational efficiency with representational power.
- **Fine-Tuning:** Embeddings are allowed to adjust during training, enabling the model to learn sentiment-specific variations in word meanings (e.g., words like "lit" or "fire" in positive contexts).

Hidden Layers:

- **Dense Layers:** Use 2–3 hidden dense layers, each with ReLU activation to introduce

non-linearity and capture complex patterns in text.

- **Dimensionality:** Layer dimensions could start at 256 neurons and reduce gradually in subsequent layers (e.g., $256 \rightarrow 128 \rightarrow 64$), allowing for a funneling effect that consolidates the learned features.
- **Dropout:** Apply dropout with rates around 0.3–0.5 to mitigate overfitting by preventing reliance on specific neurons during training.
- **Regularization:** L2 regularization on dense layers is applied to reduce overfitting, particularly important given the relatively small size and variability of labeled sentiment data.

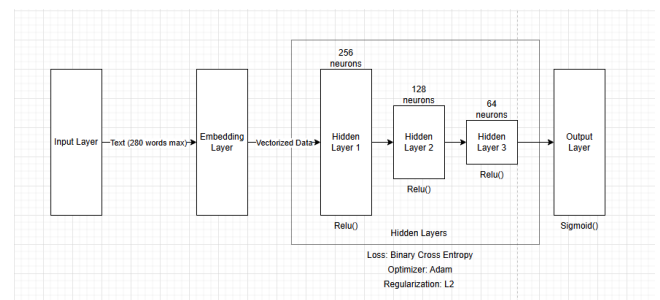
Output Layer:

- **Structure:** For binary sentiment classification (positive vs. negative), a single neuron with sigmoid activation is used, producing a probability score between 0 and 1, representing the likelihood of positive sentiment.
- **Loss Function:** Given the binary classification nature:
 - **Binary Cross-Entropy Loss:** Suitable for binary classification, this function calculates the log loss for positive and negative samples, penalizing incorrect predictions more heavily. This ensures the model optimizes for accurate sentiment predictions.
 - **Weighted Loss:** In case of class imbalance (e.g., more positive tweets than negative or neutral), class weights can be adjusted within the loss function to penalize minority classes more, improving model performance on underrepresented sentiments.

Optimizer:

- **Adam Optimizer:** Adam is chosen for its adaptive learning rate properties, which accelerate convergence. The initial learning rate is set around 0.001, with potential decay to fine-tune convergence.

Model Diagram:



3 Customizations and Optimizations

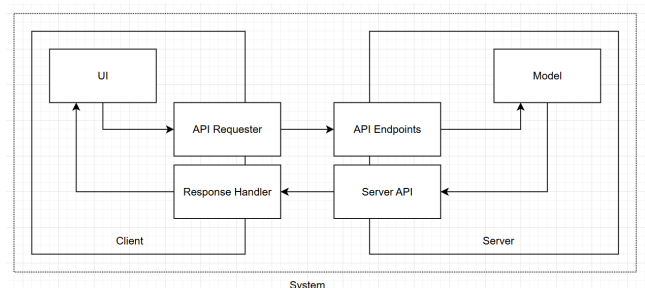
- **Class Weights:** By assigning higher weights to minority classes (e.g., negative sentiment if it's underrepresented), the model can better generalize and avoid bias toward majority classes, an essential adjustment for accurate sentiment predictions.
- **Regularization & Dropout:** L2 regularization on weights and dropout rates between 0.3–0.5 on hidden layers help manage overfitting, which is common in sentiment analysis due to diverse, opinion-heavy language in tweets.

4 Utility Application Design

4.1. Application Overview

- The utility application will serve as an interface to interact with the trained sentiment analysis model, accepting text inputs (tweets) and returning sentiment predictions.
- The model will run on a server, while the client application interacts via an API.

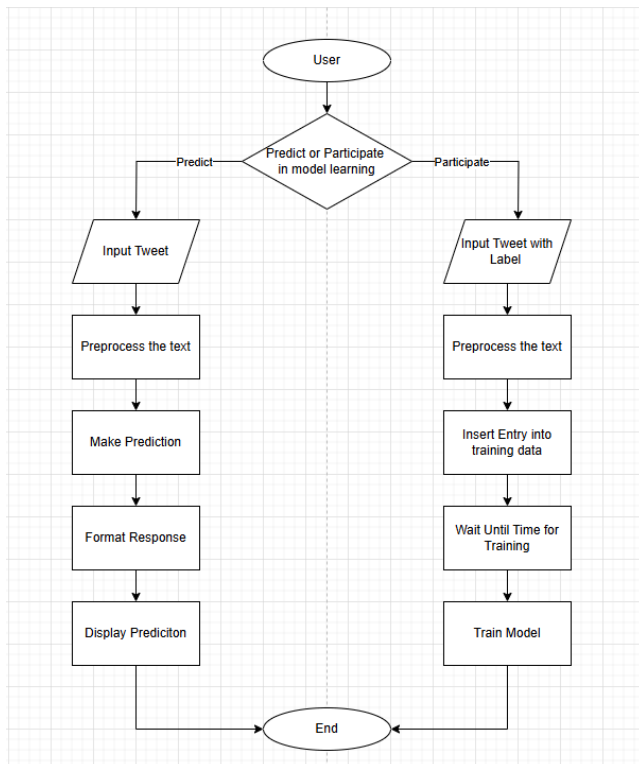
4.2. Architecture



- **Client-Server Model:**

- Server: Hosts the MLP model, API endpoints, and handles prediction requests.
- Client: User interface for inputting tweets and viewing predictions.
- **API Endpoints:**
 - /predict: Accepts tweet input and returns a sentiment prediction (JSON format).
 - /health: Health check endpoint to ensure the server is running.
- **Data Flow:**
 - The client sends a POST request with tweet text to the /predict endpoint.
 - The server processes the tweet, generates predictions, and sends a JSON response back to the client.

4.3. Interaction Flow



- **Choose Use Case:** Pick between getting a prediction or feeding the model.
- **Input:** The user inputs the data to the model through the UI
- **Preprocessing:** Ensure tweets are limited to 280 characters, filter non-text content or URLs if necessary, and use word embedding techniques on the input text.
- **Prediction and Output:**

- The server processes the input, runs the model and returns a probability score for each sentiment class.
- Display the predicted sentiment (e.g., Positive, Negative, Neutral) along with confidence levels.

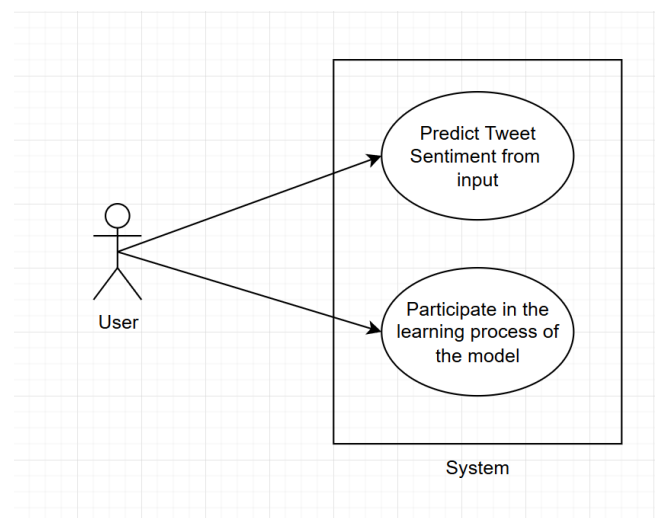
- **Learning from New Data:**

- The server will process the input entry along with the label and add it to the training data.
- After a set amount of time or new inputs, the server will refit the model on the new data.

4.4. Scalability and Deployment

- **Containerization:** Use Docker to package the model and application for easy deployment.
- **Scalability:** Deploy on a cloud platform (e.g., AWS, GCP) with auto-scaling capabilities to handle fluctuations in traffic.
- **Security:** Implement authentication for API access, especially if the application will be publicly accessible.

4.5. Use Cases



- **Actors:** The potential users for the model.
- **Actions:** The user can either get a prediction from the system according to their input or provide input with a label to help the model learn from new data.